

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT & CÔNG NGHỆ**

Trí tuệ - Sáng tạo - Năng động - Đổi mới

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG RAG HỖ TRỢ TRA CỨU
PHÁP LUẬT GIAO THÔNG VIỆT NAM
(GIẢI THÍCH KIẾN TRÚC, THUẬT TOÁN VÀ CƠ CHẾ VẬN HÀNH)**

Chuyên ngành: Công nghệ thông tin

Ngành: Công nghệ thông tin

Sinh viên thực hiện: Quách Trương Phúc

Người hướng dẫn: [Tên giảng viên hướng dẫn]

Cần Thơ, tháng 8 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng Khóa luận này là do chính tôi thực hiện, không sao chép dưới bất kỳ hình thức nào hay thuê hoặc nhờ người khác thực hiện.

Dữ liệu và kết quả phân tích trong Khóa luận đảm bảo tính chính xác, khách quan và trung thực, không có bất kỳ sự ngụy tạo và điều chỉnh kết quả nghiên cứu bằng sự chủ quan của tác giả.

Cần Thơ, ngày ... tháng ... năm 2026

Tác giả

(Ký tên và ghi rõ họ và tên)

Quách Trương Phúc

Lời Cảm Ơn

Tác giả xin gửi lời cảm ơn chân thành đến [Tên giảng viên hướng dẫn] đã tận tình hướng dẫn trong quá trình thực hiện khóa luận này. Tác giả cũng xin cảm ơn các thầy cô trong Khoa Kỹ thuật & Công nghệ, Trường Đại học Tây Đô, đã tạo điều kiện thuận lợi cho việc học tập và nghiên cứu.

Tóm Tắt

Khóa luận này trình bày việc xây dựng hệ thống RAG (Retrieval-Augmented Generation) hỗ trợ tra cứu pháp luật giao thông Việt Nam. Hệ thống kết hợp truy xuất lai (hybrid retrieval) giữa dense retrieval bằng vector embedding và BM25, sử dụng LangGraph state machine để điều phối luồng xử lý, và Google Gemini 2.5 Flash làm mô hình ngôn ngữ chính. Hệ thống đạt Citation Correctness tối thiểu 0.85 trên gold set 30 câu hỏi, xác minh trích dẫn nghiêm ngặt trước khi trả kết quả cho người dùng.

Abstract

This thesis presents a RAG (Retrieval-Augmented Generation) system for Vietnamese traffic law retrieval. The system combines hybrid retrieval between dense vector embedding and BM25, uses LangGraph state machine for workflow orchestration, and Google Gemini 2.5 Flash as the primary language model. The system achieves Citation Correctness of at least 0.85 on a 30-question gold set, with strict citation verification before returning results to users.

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Abstract	iii
Danh mục ký hiệu và từ viết tắt	vi
1 Tổng Quan Hệ Thống	1
2 Kiến Thức Nền Tảng	2
2.1 RAG: Truy Xuất và Sinh Tăng Cường	2
2.2 LLM: Mô Hình Ngôn Ngữ Lớn	2
2.3 Embedding và Vector Database	3
2.4 Chunking: Phân Mảnh Văn Bản	3
2.5 TF-IDF và BM25	4
2.6 LangGraph và State Machine	5
2.7 Agent trong Ngữ Cảnh RAG	5
2.8 HITL: Con Người Trong Vòng Lặp	5
2.9 Streaming qua SSE và Cơ Chế Buffer-Validate	5
2.10 Corpus	6
2.11 Worker: Đơn Vị Xử Lý Đồng Thời	6
2.12 Citation và Citation Correctness	7
2.13 Gold Set	7
3 Kiến Trúc Hệ Thống	8
4 LangGraph State Machine	9
4.1 Khái niệm	9
4.2 Định nghĩa AgentState	9
4.3 Các nút xử lý	9
4.4 Cơ chế loop guard	10
4.5 Buffer-validate-stream	10
5 Hybrid Retrieval	11
5.1 Dense Retrieval	11
5.2 BM25 Okapi	11

5.3	Vấn đề bất thường tần suất từ	12
5.4	Reciprocal Rank Fusion	12
5.5	Quy trình hybrid retrieval	13
6	Tiền Xử Lý Văn Bản Tiếng Việt và Phân Mảnh	14
6.1	Chuẩn hóa văn bản (VietnameseNormalizer)	14
6.2	Phân mảnh văn bản (Custom Chunker)	14
7	LLM Integration	15
8	Citation Pipeline	16
9	Web Search Fallback	17
10	Đánh Giá Hệ Thống RAGAS-lite [10]	18
11	Bảo Mật và PDPA	19
12	Domain Model	20
13	Concurrency và Thread Safety	21
14	Ingest Pipeline	22
15	Ràng Buộc Phần Cứng	23
16	Kiến Trúc Giao Diện	24
17	Giao Diện API	25
18	Kiến Trúc Kiểm Thử	26
19	Tổng Kết	27

Danh Mục Ký Hiệu và Từ Viết Tắt

AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface
BM25	Best Match 25 (thuật toán xếp hạng văn bản)
CI/CD	Continuous Integration / Continuous Deployment
DDG	DuckDuckGo
HITL	Human-in-the-Loop
LLM	Large Language Model (Mô hình ngôn ngữ lớn)
MVP	Minimum Viable Product
PDPA	Personal Data Protection Act (Nghị định 13/2023/NĐ-CP [3])
RAG	Retrieval-Augmented Generation
RPM	Requests Per Minute
RPD	Requests Per Day
RRF	Reciprocal Rank Fusion
SDLC	Software Development Life Cycle
SSE	Server-Sent Events
TPM	Tokens Per Minute
WAL	Write-Ahead Log

Chương 1

Tổng Quan Hệ Thống

Hệ thống RAG hỗ trợ tra cứu pháp luật giao thông Việt Nam là một giải pháp công nghệ được xây dựng trên nền tảng trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên, nhằm giải quyết bài toán tra cứu văn bản quy phạm pháp luật giao thông đường bộ. Hệ thống giải quyết bốn hạn chế cốt lõi của phương pháp tra cứu truyền thống: người dùng phải đọc thủ công các tệp PDF dài hàng chục đến hàng trăm trang, các mô hình ngôn ngữ lớn phổ dụng như ChatGPT thường bịa đặt thông tin (hallucination) về số tiền phạt và điều khoản không tồn tại, văn bản pháp luật thay đổi liên tục khiến người dân không nắm được bản mới nhất, và ngôn ngữ pháp lý phức tạp gây khó hiểu cho người không chuyên.

Hệ thống kết hợp kiến trúc truy xuất thông tin lai (hybrid retrieval) giữa tìm kiếm ngữ nghĩa bằng vector embedding và tìm kiếm từ khóa truyền thống BM25, đồng thời áp dụng quy trình xác minh trích dẫn nghiêm ngặt trước khi trả kết quả cho người dùng. Mỗi câu trả lời đều kèm trích dẫn có cấu trúc đến từng văn bản, Điều, Khoản, Điểm, cùng trạng thái hiệu lực cụ thể. Hệ thống vận hành trên kiến trúc đa tầng với FastAPI làm khung API, LangGraph làm bộ điều phối trạng thái cho luồng xử lý tự động, ChromaDB [12] làm cơ sở dữ liệu vector, và Google Gemini 2.5 Flash làm mô hình ngôn ngữ chính.

Hệ thống RAG hỗ trợ tra cứu pháp luật giao thông Việt Nam là một giải pháp công nghệ được xây dựng trên nền tảng trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên, nhằm giải quyết bài toán tra cứu văn bản quy phạm pháp luật giao thông đường bộ. Hệ thống giải quyết bốn hạn chế cốt lõi của phương pháp tra cứu truyền thống: người dùng phải đọc thủ công các tệp PDF dài hàng chục đến hàng trăm trang, các mô hình ngôn ngữ lớn phổ dụng như ChatGPT thường bịa đặt thông tin (hallucination) về số tiền phạt và điều khoản không tồn tại, văn bản pháp luật thay đổi liên tục khiến người dân không nắm được bản mới nhất, và ngôn ngữ pháp lý phức tạp gây khó hiểu cho người không chuyên.

Hệ thống kết hợp kiến trúc truy xuất thông tin lai (hybrid retrieval) giữa tìm kiếm ngữ nghĩa bằng vector embedding và tìm kiếm từ khóa truyền thống BM25, đồng thời áp dụng quy trình xác minh trích dẫn nghiêm ngặt trước khi trả kết quả cho người dùng. Mỗi câu trả lời đều kèm trích dẫn có cấu trúc đến từng văn bản, Điều, Khoản, Điểm, cùng trạng thái hiệu lực cụ thể. Hệ thống vận hành trên kiến trúc đa tầng với FastAPI làm khung API, LangGraph làm bộ điều phối trạng thái cho luồng xử lý tự động, ChromaDB làm cơ sở dữ liệu vector, và Google Gemini 2.5 Flash làm mô hình ngôn ngữ chính.

Chương 2

Kiến Thức Nền Tảng

Phần này giải thích các khái niệm cốt lõi mà hệ thống xây dựng trên đó. Mỗi khái niệm được trình bày từ định nghĩa đến vai trò cụ thể trong dự án.

2.1 RAG: Truy Xuất và Sinh Tăng Cường

RAG (Retrieval-Augmented Generation) là kiến trúc [5] kết hợp hai bước trong xử lý câu hỏi. Bước truy xuất (retrieval) tìm kiếm các đoạn văn bản liên quan từ một kho tài liệu có sẵn. Bước sinh (generation) đưa các đoạn văn bản vừa tìm được vào một mô hình ngôn ngữ lớn để tạo câu trả lời. LLM trong RAG không dựa vào trí nhớ nội tại của nó. Nó bị ràng buộc chỉ được sử dụng thông tin từ các đoạn văn bản được cung cấp.

LLM phổ dụng như ChatGPT vận hành khác. Nó sinh câu trả lời từ toàn bộ tri thức đã học trong quá trình huấn luyện, không có cơ chế kiểm tra xem thông tin đó có chính xác hay không. Với câu hỏi pháp luật Việt Nam, ChatGPT bịa ra số tiền phạt không tồn tại, điều khoản không tồn tại, hoặc trích dẫn văn bản đã hết hiệu lực. RAG giải quyết vấn đề này bằng cách tách biệt kho tri thức (corpus văn bản luật) khỏi khả năng diễn đạt (LLM). Kho tri thức có thể được kiểm tra, cập nhật, và kiểm soát độc lập với LLM.

Trong dự án này, kho tri thức là 30 văn bản pháp luật giao thông đã được parse, chunk, embed và lưu vào ChromaDB. LLM là Gemini 2.5 Flash. Khi người dùng hỏi "Vượt đèn đỏ phạt bao nhiêu?", hệ thống tìm các chunk liên quan trong ChromaDB, đưa chúng vào prompt cùng câu hỏi, và yêu cầu Gemini chỉ trả lời dựa trên những chunk đó. Gemini không được phép dùng kiến thức bên ngoài.

Agentic RAG là bước tiến xa hơn. Thay vì một pipeline tuyến tính cố định, một Agent dùng LangGraph tự quyết định luồng xử lý. Agent có thể viết lại câu hỏi nếu lượt tìm đầu tiên không tốt. Agent có thể chấm điểm chất lượng kết quả tìm được và quyết định có tìm lại hay không. Agent có thể chuyển sang tìm trên web nếu kho nội bộ không có câu trả lời. Trong dự án này, Agent được cài đặt bằng LangGraph state machine với sáu nút xử lý và ba bộ đếm loop guard.

2.2 LLM: Mô Hình Ngôn Ngữ Lớn

Mô hình ngôn ngữ lớn (Large Language Model) là một mạng nơ-ron được huấn luyện trên hàng tỷ câu văn bản để dự đoán từ tiếp theo trong một chuỗi. Kết quả của quá trình huấn luyện này là một mô hình có khả năng hiểu và sinh văn bản tiếng người ở mức độ đáng kinh ngạc. LLM không "biết" sự thật. Nó dự đoán chuỗi từ có xác suất cao nhất dựa trên mẫu đã học.

Dự án sử dụng hai LLM. Gemini 2.5 Flash của Google [11] làm LLM chính cho toàn bộ luồng xử lý: rewrite câu hỏi, grade chất lượng retrieval, generate câu trả lời. Mô hình này có free tier 250 request mỗi ngày, cửa sổ ngữ cảnh 1 triệu token (tương đương khoảng 750.000 từ), và chất lượng tiếng Việt tốt. OpenAI GPT-4o-mini làm LLM phụ, chỉ dùng cho

evaluation. Lý do tách biệt: nếu cùng một LLM vừa sinh câu trả lời vừa chấm điểm câu trả lời đó, điểm số sẽ thiên vị (circularity). GPT-4o-mini có khả năng xuất JSON có cấu trúc ổn định, phù hợp cho vai trò judge.

LLM không chạy cục bộ trên máy phát triển. GPU MX330 2GB VRAM không đủ để chạy bất kỳ LLM nào có chất lượng chấp nhận được cho legal reasoning. Toàn bộ LLM inference được gọi qua API cloud.

2.3 Embedding và Vector Database

Embedding là quá trình biến một đoạn văn bản thành một vector số thực nhiều chiều. Vector này nằm trong một không gian mà khoảng cách giữa hai vector phản ánh mức độ tương đồng ngữ nghĩa giữa hai đoạn văn bản gốc. Hai câu có nghĩa giống nhau sẽ có hai vector gần nhau. Hai câu về chủ đề khác nhau sẽ có hai vector xa nhau.

Hệ thống dùng mô hình intfloat/multilingual-e5-small [8] để thực hiện embedding. Mô hình này nhận văn bản đầu vào và trả về một vector 384 chiều. Kích thước 384 chiều là sự cân bằng giữa chất lượng biểu diễn ngữ nghĩa và hiệu năng tính toán: vector càng nhiều chiều càng biểu diễn tốt nhưng càng tốn bộ nhớ và thời gian so sánh. Mô hình này hỗ trợ đa ngôn ngữ, bao gồm tiếng Việt.

Yêu cầu kỹ thuật then chốt của e5-small là tiền tố. Khi embedding một câu hỏi, văn bản phải có tiền tố "query: "ở đầu. Khi embedding một đoạn văn bản tài liệu, văn bản phải có tiền tố "passage: "ở đầu. Thiếu tiền tố này, chất lượng truy xuất giảm 20 đến 40 phần trăm. Đây là lỗi âm thầm: không có exception, không có crash, chỉ có kết quả tìm kiếm kém đi mà không ai phát hiện.

Vector database là cơ sở dữ liệu được thiết kế để lưu trữ và tìm kiếm vector. Khác với cơ sở dữ liệu quan hệ tìm kiếm theo giá trị chính xác, vector database tìm kiếm theo khoảng cách: "tìm 10 vector gần nhất với vector truy vấn". Hệ thống dùng ChromaDB, một vector database nhẹ chạy local không cần Docker. Mỗi bản ghi trong ChromaDB gồm ba thành phần: một định danh (id), văn bản gốc (document), và vector embedding 384 chiều. Kèm theo là metadata chứa thông tin về Điều, Khoản, Điểm, trạng thái hiệu lực, và đường dẫn đến văn bản gốc.

2.4 Chunking: Phân Mảnh Văn Bản

Văn bản pháp luật thường dài. Nghị định 168/2024 dài 84 trang. Không thể nhét toàn bộ 84 trang này vào prompt của LLM, dù Gemini có ngưỡng 1 triệu token. Việc này vừa tốn token (tốn tiền nếu dùng API trả phí), vừa làm loãng thông tin quan trọng trong một biển văn bản không liên quan.

Chunking giải quyết vấn đề này bằng cách cắt văn bản dài thành các đoạn nhỏ (chunk). Hệ thống chỉ chọn những chunk liên quan nhất với câu hỏi để đưa vào LLM. Với câu hỏi về vượt đèn đỏ, chỉ chunk chứa Điều 6 Nghị định 168 là cần thiết. 83 trang còn lại bị bỏ qua.

Chunking không đơn giản là cắt theo số ký tự cố định. Văn bản pháp luật có cấu trúc phân cấp: Chương chứa Mục, Mục chứa Điều, Điều chứa Khoản, Khoản chứa Điểm. Cắt ngang giữa một Điều sẽ làm mất ngữ cảnh pháp lý. Dự án có ba custom chunker cho ba loại văn bản. Chunker cho Luật nhận diện Chương, Mục, Điều. Chunker cho Nghị định

nhận diện Chương, Điều, Khoản, Điểm. Chunker cho Thông tư nhận diện Điều, Khoản. Mỗi chunk tương ứng với một đơn vị pháp lý hoàn chỉnh (thường là một Điều hoặc một Khoản) và mang theo metadata hierarchy để phục vụ trích dẫn sau này.

2.5 TF-IDF và BM25

TF-IDF (Term Frequency - Inverse Document Frequency) là nền tảng của tìm kiếm văn bản. Ý tưởng cốt lõi: một từ quan trọng nếu nó xuất hiện nhiều trong một văn bản cụ thể (TF cao) nhưng xuất hiện ít trong toàn bộ tập văn bản (IDF cao). Từ "và", "của", "là" có TF cao nhưng IDF thấp vì xuất hiện trong mọi văn bản - chúng không có giá trị phân biệt. Từ "nghị định", "điều", "khoản" xuất hiện trong mọi văn bản luật - IDF của chúng cũng thấp. Từ "vượt đèn đỏ", "nồng độ cồn", "xe máy" xuất hiện ít hơn - IDF của chúng cao hơn.

BM25 Okapi cải tiến TF-IDF [7] bằng hai cơ chế. Thứ nhất, nó giới hạn ảnh hưởng của TF: một từ xuất hiện 100 lần không quan trọng gấp 100 lần từ xuất hiện 1 lần. Hàm bão hòa $TF/(TF + k_1)$ đảm bảo sau một ngưỡng, tần suất tăng thêm không làm tăng điểm đáng kể. Thứ hai, nó điều chỉnh theo độ dài văn bản: một từ xuất hiện 5 lần trong văn bản 100 từ quan trọng hơn 5 lần trong văn bản 10.000 từ. Tham số b kiểm soát mức độ phạt văn bản dài.

Hàm điểm số BM25 cho truy vấn Q gồm các từ $q_1, q_2, \dots, q_n$ trên văn bản D được biểu diễn như sau:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (2.1)$$

Trong đó $\text{IDF}(q_i)$ được tính bằng công thức:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (2.2)$$

với N là tổng số văn bản trong corpus và $n(q_i)$ là số văn bản chứa từ q_i . Công thức IDF đảm bảo các từ hiếm gặp có trọng số cao hơn các từ phổ biến.

Các tham số của công thức (2.1):

- $f(q_i, D)$: tần suất xuất hiện của từ q_i trong văn bản D .
- k_1 : tham số bão hòa tần suất từ, mặc định 1.2. Khi k_1 càng nhỏ, ảnh hưởng của tần suất từ càng bị giới hạn sớm.
- b : tham số điều chỉnh theo độ dài văn bản, mặc định 0.75. Khi b càng gần 1, mức phạt văn bản dài càng nặng.
- $|D|$: độ dài của văn bản D tính bằng số từ; avgdl: độ dài trung bình của toàn bộ văn bản trong corpus.

Trong dự án, BM25 được cài đặt qua thư viện rank_bm25 với tokenize tiếng Việt bằng pyvi. pyvi giải quyết vấn đề đặc thù của tiếng Việt: đây là ngôn ngữ đơn lập, từ ghép gồm nhiều âm tiết viết rời ("giao thông" là một từ nhưng viết bằng hai âm tiết có khoảng trắng). Tách từ bằng khoảng trắng đơn thuần sẽ biến "giao thông" thành hai từ "giao" và "thông", làm mất nghĩa. pyvi dùng từ điển để nhận diện ranh giới từ chính xác.

2.6 LangGraph và State Machine

LangGraph là thư viện xây dựng [9] ứng dụng agent dưới dạng đồ thị trạng thái. Một đồ thị gồm các nút (node) và các cạnh (edge). Mỗi nút thực hiện một thao tác cụ thể. Các cạnh xác định thứ tự thực hiện. Trạng thái (state) là một đối tượng dùng chung, được truyền từ nút này sang nút khác, tích lũy dữ liệu qua từng bước xử lý.

Khác với pipeline tuyến tính thông thường (A -> B -> C -> D), LangGraph cho phép các cạnh có điều kiện. Sau nút grade, hệ thống có thể đi đến generate (nếu điểm cao), quay lại rewrite (nếu điểm trung bình), hoặc chuyển sang web_search (nếu điểm thấp). Quyết định này dựa trên giá trị của relevance_score và iterations trong state tại thời điểm đó.

LangGraph cũng cung cấp cơ chế persistence. Toàn bộ state có thể được lưu vào SQLite sau mỗi bước, cho phép truy vết toàn bộ quá trình xử lý của một câu hỏi để debug hoặc audit. Trong MVP, persistence được dùng để lưu lịch sử hội thoại, không dùng để resume phiên bị gián đoạn.

2.7 Agent trong Ngữ Cảnh RAG

Trong ngữ cảnh hệ thống RAG, Agent là thành phần tự động đưa ra quyết định về luồng xử lý. Agent không phải là một thực thể riêng biệt. Nó là tập hợp của LangGraph state machine, các nút xử lý, và các điều kiện rẽ nhánh.

Agent khác với pipeline RAG đơn giản ở chỗ nó có vòng lặp. Pipeline RAG đơn giản: nhận câu hỏi -> tìm kiếm -> sinh câu trả lời -> trả về. Agent RAG: nhận câu hỏi -> viết lại câu hỏi -> tìm kiếm -> chấm điểm -> nếu điểm thấp thì viết lại câu hỏi và tìm lại -> nếu vẫn thấp thì chuyển sang web -> sinh câu trả lời -> kiểm tra trích dẫn -> nếu sai thì sinh lại. Agent có khả năng tự sửa sai và thích nghi với chất lượng kết quả tìm kiếm.

2.8 HITL: Con Người Trong Vòng Lặp

HITL (Human-in-the-Loop) là cơ chế đưa con người vào quy trình tự động tại các điểm quyết định quan trọng. Khi hệ thống không chắc chắn về câu trả lời, nó tạm dừng và chờ con người phê duyệt thay vì tự động trả về kết quả có thể sai.

Trong thiết kế đầy đủ của dự án, HITL được kích hoạt khi hệ thống phải dùng web search. Lý do: kết quả web search không được kiểm soát như corpus nội bộ. Một trang web có thể chứa thông tin lỗi thời, sai lệch, hoặc không chính thống. Trước khi trả câu trả lời dựa trên web cho người dùng, hệ thống gửi yêu cầu phê duyệt đến Admin. Admin xem xét các kết quả web, quyết định Approve hoặc Reject.

Trong MVP, HITL đã bị tắt. Lý do: độ phức tạp triển khai cao. HITL yêu cầu cơ chế interrupt trong LangGraph, giao diện Admin riêng, và ánh xạ thread giữa user session và admin session. Thay vào đó, MVP gắn disclaimer vào mọi câu trả lời từ web: "Nguồn: Web (chưa qua kiểm duyệt). Vui lòng xác minh trước khi áp dụng." HITL được giữ lại trong thiết kế như một hướng phát triển tương lai.

2.9 Streaming qua SSE và Cơ Chế Buffer-Validate

SSE (Server-Sent Events) là giao thức cho phép server đẩy dữ liệu về client theo thời gian thực qua một kết nối HTTP duy trì. Khác với WebSocket là hai chiều, SSE chỉ một chiều từ server đến client. Client mở một kết nối HTTP GET và server giữ kết nối này mở,

gửi các sự kiện khi có dữ liệu mới.

Trong dự án, SSE được dùng để stream câu trả lời từ backend về frontend. Người dùng thấy câu trả lời xuất hiện dần từng phần.

Streaming trực tiếp từ LLM đặt ra thách thức cho citation validation. Nếu câu trả lời được stream từng phần ra giao diện, rồi phát hiện một trích dẫn sai ở phần sau, hệ thống không thể rút lại phần đã gửi đến trình duyệt. Người dùng sẽ thấy câu trả lời thay đổi giữa chừng.

Giải pháp là buffer-validate-stream. Hệ thống không stream trực tiếp từ LLM. Thay vào đó: LLM sinh toàn bộ câu trả lời vào bộ đệm trong bộ nhớ (buffer), CitationValidator kiểm tra toàn bộ citation trong bộ đệm (validate), và chỉ khi tất cả citation hợp lệ hoặc đã hết số lần thử sinh lại ($\text{regen_count} \geq 2$), câu trả lời hoàn chỉnh mới được stream ra giao diện qua SSE (stream). Cơ chế này đánh đổi latency - người dùng phải đợi đến khi validate xong mới thấy chữ đầu tiên - để lấy sự chính xác và nhất quán tuyệt đối của câu trả lời.

2.10 Corpus

Corpus là tập hợp toàn bộ văn bản pháp luật mà hệ thống có quyền tra cứu. Trong dự án này, corpus ban đầu gồm 30 văn bản pháp luật giao thông đường bộ Việt Nam, bao gồm: Nghị định 168/2024/NĐ-CP [1] (xử phạt vi phạm hành chính giao thông đường bộ), Luật Trật tự An toàn Giao thông Đường bộ 36/2024/QH15 [2], các Thông tư hướng dẫn, và các văn bản liên quan khác.

Corpus khác với database thông thường. Database lưu dữ liệu có cấu trúc (bảng, cột, khóa). Corpus lưu văn bản phi cấu trúc đã được xử lý thành chunks có embedding. Mỗi chunk trong corpus là một đơn vị có thể được tìm kiếm và trích xuất độc lập.

Corpus được xây dựng qua ingest pipeline. PDF được tải về từ các nguồn công khai (thuvienphapluat.vn, phapdien.moj.gov.vn), parse bằng PyMuPDF, chunk bằng custom chunker, embed bằng multilingual-e5-small, và lưu vào ChromaDB. BM25 index được xây dựng từ toàn bộ văn bản đã chunk. Kết quả là corpus có thể được truy vấn bằng cả hai phương pháp dense và sparse.

Chất lượng corpus quyết định chất lượng toàn bộ hệ thống. Nếu PDF parse sai, chunk bị cắt lệch, hoặc embedding bị thiếu prefix, câu trả lời sẽ sai bất kể LLM có tốt đến đâu. Nguyên tắc của dự án: corpus quality hơn quantity. 30 văn bản sạch, được parse chính xác, tốt hơn 100 văn bản parse lỗi.

Corpus cần được bảo trì liên tục. Văn bản pháp luật thay đổi: văn bản mới được ban hành, văn bản cũ hết hiệu lực. Script ingest_corpus.py thêm văn bản mới mà không xóa văn bản cũ (append mode). Script check_expired.py quét corpus định kỳ để phát hiện văn bản đã hoặc sắp hết hiệu lực. Văn bản hết hiệu lực không bị xóa khỏi corpus vì vẫn có thể hữu ích cho tra cứu lịch sử, nhưng citation của chúng được gắn cảnh báo.

2.11 Worker: Đơn Vị Xử Lý Đồng Thời

Worker là một tiến trình hệ điều hành chạy ứng dụng FastAPI. Mỗi worker có thể nhận và xử lý một HTTP request tại một thời điểm. FastAPI chạy trên Uvicorn, một ASGI server. Uvicorn mặc định chạy 1 worker. Tuy nhiên, convention phổ biến khi triển khai FastAPI là dùng $\text{workers} = (2 \times \text{CPU}) + 1$, tạo ra 13 worker trên máy i5-1035G1 4

nhân. Với cấu hình này, 13 tiến trình cùng truy cập ChromaDB.

Hệ thống chạy với chính xác 1 worker. Lý do: ChromaDB, vector database dùng trong dự án, sử dụng SQLite nội bộ. SQLite không được thiết kế để nhiều tiến trình ghi đồng thời. Khi 13 worker cùng truy cập ChromaDB, SQLite bị treo (busy timeout), sinh lỗi segmentation fault, hoặc hỏng vĩnh viễn tệp cơ sở dữ liệu. Đây là vấn đề đã được ghi nhận rộng rãi trong cộng đồng ChromaDB.

Giải pháp single-worker đánh đổi throughput lấy tính toàn vẹn dữ liệu. Hệ thống xử lý được khoảng 2.5 câu hỏi mỗi phút thay vì 13 câu mỗi phút nếu dùng multi-worker. Tuy nhiên, với quy mô dự án khóa luận (một người dùng, dưới 62 câu mỗi ngày do giới hạn Gemini free tier), single-worker không phải là nút thắt cổ chai. Thêm vào đó, LLM inference qua API cloud chiếm phần lớn thời gian xử lý, không phải CPU, nên nhiều worker cũng không cải thiện đáng kể throughput.

2.12 Citation và Citation Correctness

Citation là trích dẫn có cấu trúc đến nguồn pháp lý. Một citation đầy đủ gồm: tên văn bản, số Điều, số Khoản, Điểm (nếu có), và trạng thái hiệu lực. Ví dụ: Nghị định 168/2024/NĐ-CP, Điều 6, Khoản 2, Điểm a (còn hiệu lực).

Citation Correctness là chỉ số đo tỷ lệ câu trả lời có ít nhất một trích dẫn khớp chính xác với đáp án chuẩn. Khớp chính xác nghĩa là doc_id, dieu, khoan, diem trong câu trả lời trùng khớp với gold set. Đây là headline metric - chỉ số quan trọng nhất của toàn bộ dự án. Mục tiêu: Citation Correctness đạt tối thiểu 0.85 trên gold set 30 câu.

Citation Correctness được tính bằng thuật toán xác định, không cần LLM judge. Điều này giúp chỉ số khách quan, có thể tái lập, và không bị ảnh hưởng bởi chất lượng của LLM judge. Hàm tính duyệt qua từng citation trong câu trả lời, so sánh doc_id, dieu, khoan, diem với gold set. Nếu tìm thấy ít nhất một citation khớp hoàn toàn, điểm là 1.0. Nếu không, điểm là 0.0.

2.13 Gold Set

Gold set là tập hợp các câu hỏi kiểm thử kèm đáp án chuẩn. Mỗi câu trong gold set chứa: câu hỏi (query), câu trả lời mong đợi (expected_answer), danh sách citation chính xác (expected_citations), và loại câu hỏi (trong corpus / ngoài corpus / adversarial).

Dự án xây dựng gold set 30 đến 50 câu. Trong đó có câu hỏi đơn giản trong corpus ("Vượt đèn đỏ phạt bao nhiêu?"), câu hỏi tình huống phức tạp ("Tôi đi xe máy va chạm với ô tô tại ngã tư không đèn, ai có lỗi?"), câu hỏi ngoài corpus ("Luật mới về xe điện năm 2026?"), và câu adversarial (câu hỏi bẫy để kiểm tra khả năng từ chối trả lời).

Gold set được đóng băng bằng hash SHA-256 để đảm bảo không bị thay đổi trong quá trình đánh giá. Điều này ngăn chặn việc "tối ưu cho bài kiểm tra chỉnh sửa gold set để điểm số đẹp hơn. Mỗi lần chạy evaluation, hệ thống kiểm tra hash của gold set trước khi thực hiện.

Chương 3

Kiến Trúc Hệ Thống

Hệ thống được tổ chức theo mô hình ba tầng (three-tier architecture). Frontend sử dụng Next.js 14 với TypeScript và Tailwind CSS để dựng giao diện trò chuyện, hỗ trợ streaming câu trả lời qua giao thức SSE. Tầng nghiệp vụ chạy trên FastAPI, chứa bộ điều phối LangGraph, tầng service và các mô hình dữ liệu Pydantic. Tầng dữ liệu bao gồm ChromaDB cho lưu trữ vector, SQLite cho lưu trữ trạng thái và lịch sử hội thoại, và hệ thống tệp cho các tệp PDF gốc, chỉ mục BM25 và bộ nhớ đệm embedding.

Kiến trúc client-server tập trung được chọn thay vì microservices vì hệ thống có quy mô nhỏ, một người quản lý được. API được thiết kế stateless, trạng thái hội thoại lưu trong SQLite thay vì trong bộ nhớ, giúp dễ dàng mở rộng và kiểm thử. Tầng agent sử dụng LangGraph với trạng thái có chủ đích, cho phép truy vết toàn bộ quá trình xử lý.

Chương 4

LangGraph State Machine

4.1 Khái niệm

LangGraph là framework xây dựng luồng xử lý dưới dạng đồ thị trạng thái (state graph). Mỗi nút (node) trong đồ thị thực hiện một bước xử lý cụ thể. Trạng thái (state) được định nghĩa là một TypedDict chứa toàn bộ dữ liệu phát sinh trong suốt vòng đời xử lý của một câu hỏi. Các nút đọc và ghi vào trạng thái này, và các cạnh có điều kiện xác định nút nào sẽ chạy tiếp theo dựa trên giá trị hiện tại của trạng thái.

4.2 Định nghĩa AgentState

AgentState là cấu trúc dữ liệu trung tâm. Nó chứa các trường sau:

Trường đầu vào: query là câu hỏi gốc từ người dùng, query_id là mã định danh duy nhất cho mỗi yêu cầu phục vụ mục đích kiểm toán, thread_id là mã định danh phiên hội thoại tùy chọn.

Trường sau bước rewrite: rewritten_query là câu hỏi đã được tối ưu, rewrite_count đếm số lần rewrite, bị chặn cứng ở mức 2.

Trường sau bước retrieve: chunks là danh sách mười chunk được truy xuất, retrieval_method ghi lại phương pháp truy xuất đã dùng (dense, bm25, hoặc hybrid).

Trường sau bước grade: relevance_score là điểm số duy nhất dùng để định tuyến, grade_explanation là giải thích của LLM.

Trường cho web fallback: web_results lưu kết quả tìm web, web_used đánh dấu đã dùng web hay chưa.

Trường sau bước generate: draft_answer và draft_citations là bản nháp trước khi kiểm duyệt.

Trường sau bước validate: answer là câu trả lời cuối cùng, citations là danh sách trích dẫn đã được xác minh, validation_warnings chứa cảnh báo cho các trích dẫn không hợp lệ, regen_count đếm số lần sinh lại bị chặn cứng ở mức 2.

Trường disclaimer và citation_statuses được thêm vào cuối mọi câu trả lời.

Trường iterations là bộ đếm vòng lặp tổng thể, bị chặn cứng ở mức 3.

4.3 Các nút xử lý

Hệ thống có sáu nút xử lý chính: rewrite, retrieve, grade, generate, web_search, và validate_citation. Thêm vào đó có các nút phụ: buffer_output, append_disclaimer, append_status, stream_sse, và refuse.

Nút rewrite nhận câu hỏi gốc và tối ưu hóa nó. Việc tối ưu này cần thiết vì câu hỏi tự nhiên thường chứa từ ngữ không chính xác hoặc thiếu ngữ cảnh. Ví dụ, câu hỏi "Vượt đèn đỏ phạt bao nhiêu tiền?" có thể được rewrite thành "Mức xử phạt hành vi không chấp hành tín hiệu đèn giao thông theo Nghị định 168/2024 là bao nhiêu?" giúp tìm kiếm chính xác hơn. Sau mỗi lần rewrite, rewrite_count tăng lên 1. Nếu rewrite_count đạt 2 mà vẫn chưa đạt kết

quả khả quan, hệ thống chuyển sang tìm web.

Nút retrieve thực hiện truy xuất lai (hybrid retrieval) trên cơ sở dữ liệu văn bản luật. Chi tiết về thuật toán được trình bày ở phần sau. Kết quả trả về mười chunk có điểm số cao nhất.

Nút grade gọi LLM để chấm điểm mức độ liên quan của các chunk vừa tìm được so với câu hỏi của người dùng. LLM trả về một điểm số từ 0 đến 1. Điểm số này là yếu tố duy nhất quyết định luồng xử lý tiếp theo. Nếu điểm từ 0.7 trở lên và số lần lặp nhỏ hơn 3, hệ thống chuyển sang nút generate. Nếu điểm từ 0.4 đến dưới 0.7 và số lần lặp nhỏ hơn 3, hệ thống quay lại nút rewrite, tăng iterations lên 1. Nếu điểm dưới 0.4, hoặc số lần lặp đã đạt 3, hoặc rewrite_count đã đạt 2, hệ thống chuyển sang nút web_search.

Nút generate nhận câu hỏi gốc và mười chunk liên quan nhất, gọi LLM với Legal Reasoning Prompt. Prompt này gồm 14 quy tắc bắt buộc, bao gồm chỉ sử dụng thông tin từ context, mọi thông tin quan trọng phải có trích dẫn, tuyệt đối không bịa số tiền phạt hay điều khoản. Đầu ra của nút này là draft_answer và draft_citations ở dạng JSON.

Nút validate_citation kiểm tra từng trích dẫn trong draft_citations có thực sự tồn tại trong context đã cung cấp hay không. Nếu tất cả đều hợp lệ, hệ thống chuyển sang buffer_output. Nếu có trích dẫn không hợp lệ và số lần sinh lại chưa đạt 2, hệ thống gọi LLM sinh lại với prompt mạnh hơn. Nếu đã sinh lại 2 lần mà vẫn không hợp lệ, hệ thống vẫn trả về câu trả lời kèm cảnh báo.

Nút web_search được kích hoạt khi corpus nội bộ không có đủ thông tin. Nút này gọi DuckDuckGo API để tìm kiếm trên web. Nếu có kết quả, LLM sinh câu trả lời dựa trên nội dung web. Nếu không có kết quả, hệ thống từ chối trả lời.

4.4 Cơ chế loop guard

Hệ thống có ba bộ đếm loop guard. iterations là bộ đếm chính, tăng sau mỗi lần từ grade quay lại rewrite, bị chặn cứng ở mức 3. Nếu iterations đạt 3 mà vẫn chưa đạt điểm grade đủ cao, hệ thống chuyển sang web_search. rewrite_count đếm số lần rewrite, bị chặn cứng ở mức 2. regen_count đếm số lần sinh lại sau khi validate thất bại, bị chặn cứng ở mức 2.

4.5 Buffer-validate-stream

Thay vì stream câu trả lời từng phần ra giao diện rồi sau đó phải rút lại nếu phát hiện lỗi (không thể thực hiện được vì đã gửi đến trình duyệt), hệ thống áp dụng cơ chế buffer-validate-stream. Toàn bộ câu trả lời được sinh ra và kiểm duyệt trong bộ nhớ trước. Chỉ sau khi vượt qua tất cả các bước kiểm duyệt, câu trả lời hoàn chỉnh mới được stream ra giao diện qua SSE.

Chương 5

Hybrid Retrieval

Truy xuất lại là kỹ thuật kết hợp hai phương pháp tìm kiếm khác nhau: tìm kiếm ngữ nghĩa bằng vector embedding (dense retrieval) và tìm kiếm từ khóa bằng thuật toán BM25 (sparse retrieval). Mỗi phương pháp có điểm mạnh và điểm yếu riêng, và việc kết hợp chúng cho kết quả tốt hơn từng phương pháp riêng lẻ, đặc biệt trên dữ liệu tiếng Việt pháp luật.

5.1 Dense Retrieval

Dense retrieval biến mỗi đoạn văn bản thành một vector số thực 384 chiều bằng mô hình embedding. Khi người dùng gửi câu hỏi, hệ thống biến câu hỏi đó thành một vector 384 chiều tương tự. Sau đó, hệ thống tính khoảng cách cosine giữa vector câu hỏi và vector của tất cả các chunk trong cơ sở dữ liệu. Chunk nào có khoảng cách gần nhất (cosine similarity cao nhất) được xem là liên quan nhất.

Hệ thống sử dụng mô hình intfloat/multilingual-e5-small làm embedding. Mô hình này có một yêu cầu kỹ thuật quan trọng: phải thêm tiền tố vào văn bản trước khi embedding. Khi embedding câu hỏi, phải thêm "query: " ở đầu. Khi embedding chunk văn bản lúc ingest, phải thêm "passage: " ở đầu. Nếu thiếu tiền tố này, chất lượng truy xuất giảm từ 20 đến 40 phần trăm, lỗi âm thầm: không có lỗi hay cảnh báo nào phát ra.

Mô hình embedding chạy trên CPU, không dùng GPU. Máy phát triển có card đồ họa MX330 với 2GB VRAM, không đủ để chạy embedding inference ổn định. Mỗi lần embedding một chunk mất khoảng 300 đến 500 mili giây.

Quy trình embedding thực hiện như sau. Một lớp cấu hình SentenceTransformer tải mô hình với tham số device="cpu". Phương thức embed_query kiểm tra xem văn bản đầu vào đã có tiền tố "query: " hay chưa, nếu chưa thì tự động thêm vào, sau đó gọi model.encode với normalize_embeddings=True để chuẩn hóa vector về độ dài đơn vị. Phương thức embed_passages thực hiện tương tự với tiền tố "passage: ".

5.2 BM25 Okapi

BM25 là thuật toán tìm kiếm từ khóa dựa trên mô hình xác suất, là phiên bản cải tiến của TF-IDF. Khác với TF-IDF chỉ tính tần suất xuất hiện của từ, BM25 tính thêm độ dài văn bản và tần suất xuất hiện tương đối để tránh thiên vị các văn bản dài.

Công thức BM25 Okapi như sau:

Hàm số điểm BM25 cho một truy vấn Q gồm các từ $q_1, q_2, \dots, q_n$ trên một văn bản D:

$$\text{score}(D, Q) = \text{tổng của } \text{IDF}(q_i) \text{ nhân với } \text{TF}(q_i, D) \text{ nhân } (k_1 \text{ cộng } 1) \text{ chia cho } (\text{TF}(q_i, D) \text{ cộng } k_1 \text{ nhân } (1 \text{ trừ } b \text{ cộng } b \text{ nhân } |D| \text{ chia } \text{avgdl}))$$

Trong đó:

$\text{IDF}(q_i)$ là logarit tự nhiên của $(N \text{ trừ } n(q_i) \text{ cộng } 0.5)$ chia $(n(q_i) \text{ cộng } 0.5)$ cộng 1. N là tổng số văn bản trong corpus. $n(q_i)$ là số văn bản chứa từ q_i . Công thức này đảm bảo các

từ hiếm gặp có trọng số cao hơn các từ phổ biến.

$TF(q_i, D)$ là số lần từ q_i xuất hiện trong văn bản D .

k_1 là tham số điều chỉnh độ bão hòa tần suất từ. Giá trị mặc định là 1.2. Khi k_1 càng nhỏ, tác động của tần suất từ càng bị giới hạn sớm.

b là tham số điều chỉnh độ dài văn bản. Giá trị mặc định là 0.75. Khi b càng gần 1, mức độ phạt văn bản dài càng lớn.

$|D|$ là độ dài của văn bản D tính bằng số từ. $avgdl$ là độ dài trung bình của tất cả văn bản trong corpus.

Hệ thống sử dụng thư viện `rank_bm25` cho cài đặt BM25. Điểm đặc biệt là `rank_bm25` chỉ hoạt động với bộ nhớ trong (in-memory), không hỗ trợ thêm dữ liệu gia tăng (incremental add). Mỗi khi có chunk mới được thêm vào, toàn bộ chỉ mục BM25 phải được xây dựng lại từ đầu. Điều này ảnh hưởng đến thiết kế ingest pipeline: sau khi thêm chunk mới vào ChromaDB, hệ thống đọc toàn bộ dữ liệu từ ChromaDB, tokenize bằng `pyvi`, xây dựng lại đối tượng `BM25Okapi`, và lưu kết quả vào tệp pickle trên đĩa.

Việc tokenize tiếng Việt cho BM25 sử dụng thư viện `pyvi`, cụ thể là hàm `ViTokenizer.tokenize`. Tiếng Việt là ngôn ngữ đơn lập, các từ ghép gồm nhiều âm tiết (ví dụ: "giao thông", "pháp luật", "điều khiển"). Việc tách từ bằng khoảng trắng đơn thuần sẽ làm mất nghĩa của các từ ghép này. `pyvi` giải quyết vấn đề này bằng cách nhận diện ranh giới từ dựa trên từ điển tiếng Việt.

5.3 Vấn đề bất thường tần suất từ

TF-IDF và BM25 gặp khó khăn với văn bản pháp luật Việt Nam vì cấu trúc lặp từ đặc thù. Một nghị định thường có các cụm từ lặp đi lặp lại như "người điều khiển xe mô tô, xe gắn máy", "phạt tiền từ X đồng đến Y đồng", "không chấp hành tín hiệu đèn giao thông". Các từ như "điều", "khoản", "điểm", "nghị định", "xe" xuất hiện với tần suất rất cao và có giá trị phân biệt thấp. Hệ thống xử lý điều này bằng cách chuẩn hóa văn bản trước khi đưa vào BM25, nhưng sự trùng lặp nội tại của văn bản luật vẫn là thách thức. RRF giúp giảm tác động này bằng cách kết hợp với dense retrieval vốn không bị ảnh hưởng bởi tần suất từ.

5.4 Reciprocal Rank Fusion

RRF là phương pháp kết hợp [6] hai danh sách xếp hạng từ hai hệ thống truy xuất khác nhau mà không cần điểm số chuẩn hóa tuyệt đối. Thay vào đó, RRF dựa trên thứ hạng (rank) của mỗi chunk trong từng danh sách.

Công thức RRF được biểu diễn như sau:

$$RRF_score(c) = \frac{1}{k + r_{dense}(c)} + \frac{1}{k + r_{bm25}(c)} \quad (5.1)$$

Trong đó $rank_dense$ là thứ hạng của chunk trong kết quả dense retrieval, $rank_bm25$ là thứ hạng của chunk trong kết quả BM25. k là hằng số, mặc định là 60.

Giá trị $k = 60$ được đề xuất bởi Cormack và cộng sự trong bài báo gốc năm 2009. k càng lớn thì trọng số của thứ hạng cao càng bị san đều, k càng nhỏ thì thứ hạng cao càng có ảnh hưởng lớn. Giá trị 60 là điểm cân bằng được thực nghiệm chứng minh hiệu quả trên nhiều bộ dữ liệu.

Công thức RRF:

$\text{RRF_score}(\text{chunk}) = 1 \text{ chia } (k \text{ cộng rank_dense}) \text{ cộng } 1 \text{ chia } (k \text{ cộng rank_bm25})$

Trong đó rank_dense là thứ hạng của chunk trong kết quả dense retrieval, rank_bm25 là thứ hạng của chunk trong kết quả BM25. k là hằng số, mặc định là 60.

Giá trị k = 60 được đề xuất bởi Cormack và cộng sự trong bài báo gốc năm 2009. k càng lớn thì trọng số của thứ hạng cao càng bị san đều, k càng nhỏ thì thứ hạng cao càng có ảnh hưởng lớn. Giá trị 60 là điểm cân bằng được thực nghiệm chứng minh hiệu quả trên nhiều bộ dữ liệu.

Ví dụ: chunk A đứng hạng 1 trong dense và hạng 5 trong BM25. Điểm RRF của chunk A là $1 \text{ chia } (60 \text{ cộng } 1) \text{ cộng } 1 \text{ chia } (60 \text{ cộng } 5) = 0.0164 \text{ cộng } 0.0154 = 0.0318$. Chunk B đứng hạng 10 trong dense và hạng 2 trong BM25. Điểm RRF của chunk B là $1 \text{ chia } (60 \text{ cộng } 10) \text{ cộng } 1 \text{ chia } (60 \text{ cộng } 2) = 0.0143 \text{ cộng } 0.0161 = 0.0304$. Chunk A thắng nhờ hạng cao hơn trong dense.

Trước khi đưa vào RRF, hệ thống chuẩn hóa điểm cosine và điểm BM25 về cùng thang đo 0-1 bằng phương pháp min-max normalization. Công thức chuẩn hóa như sau:

$\text{normalized_score} = (\text{score} - \text{min_score}) \text{ chia } (\text{max_score} - \text{min_score})$

Trước khi đưa vào RRF, hệ thống chuẩn hóa điểm cosine và điểm BM25 về cùng thang đo 0-1 bằng phương pháp min-max normalization. Công thức chuẩn hóa như sau:

$$\text{score}_{\text{norm}} = \frac{\text{score}_{\text{raw}} - \text{score}_{\text{min}}}{\text{score}_{\text{max}} - \text{score}_{\text{min}}} \quad (5.2)$$

Nếu max_score bằng min_score, tất cả các điểm được gán giá trị 0.5. Bước chuẩn hóa này ngăn chặn trường hợp một hệ thống có điểm số cao hơn nhiều so với hệ thống kia và lấn át hoàn toàn kết quả hợp nhất.

5.5 Quy trình hybrid retrieval

Khi nhận một truy vấn, HybridRetriever thực hiện các bước sau đây theo thứ tự:

Bước một: gọi EmbeddingService.embed_query để biến câu hỏi thành vector 384 chiều. Tiền tố "query: " được tự động thêm vào nếu thiếu. Bước hai: truy vấn ChromaDB collection law_chunks với vector này, yêu cầu 20 kết quả, kèm theo trường documents, metadatas, và distances. Kết quả trả về là danh sách 20 chunk cùng với khoảng cách cosine. Hệ thống chuyển khoảng cách cosine thành độ tương đồng cosine bằng công thức 1 trừ distance. Bước ba: nếu BM25 không được kích hoạt, hệ thống trả về 20 chunk đã sắp xếp theo dense score. Nếu BM25 được kích hoạt, hệ thống tiếp tục bước bốn. Bước bốn: tokenize truy vấn bằng pyvi, gọi BM25.get_scores để lấy điểm BM25 cho tất cả chunk trong corpus. Lấy 20 chunk có điểm BM25 cao nhất. Bước năm: chuẩn hóa cả dense_scores và bm25_scores về thang 0-1 bằng min-max normalization. Bước sáu: áp dụng RRF để hợp nhất hai danh sách xếp hạng. Bước bảy: sắp xếp tất cả chunk theo điểm RRF giảm dần, trả về 10 chunk có điểm cao nhất.

Chương 6

Tiền Xử Lý Văn Bản Tiếng Việt và Phân Mảnh

6.1 Chuẩn hóa văn bản (VietnameseNormalizer)

Văn bản pháp luật Việt Nam có đặc thù định dạng hỗn hợp. Các số thứ tự xuất hiện dưới nhiều dạng: "1.", "1)", "a)", "(a)". Dấu gạch ngang có ba biến thể: gạch ngắn (-), gạch dài (—), và gạch đôi (≡). Chữ hoa chữ thường biến đổi không nhất quán: "Điều", "điều", "ĐIỀU" đều xuất hiện trong cùng một văn bản. Unicode có thể ở dạng tổ hợp (NFC) hoặc phân ly (NFD). Nếu không chuẩn hóa, BM25 và embedding sẽ xử lý các biến thể này như những từ khác nhau, làm giảm chất lượng truy xuất mà không có lỗi hay cảnh báo nào.

VietnameseNormalizer xử lý bốn lớp vấn đề:

- **lowercase:** Chuẩn hóa các nhãn mục pháp luật: "Điều> "điều", "Khoản> "khoản", "Điểm> "điểm", "Chương> "chương", "Phụ lục> "phụ lục", "Mục> "mục".
- **unify_dashes:** Thay thế tất cả biến thể dấu gạch ngang (-, —, ≡) bằng dấu gạch ngắn tiêu chuẩn (-).
- **strip_punctuation:** Loại bỏ dấu câu khỏi số thứ tự: "1.", "1)", "a)" trở thành "1 ", "a " với khoảng trắng hai bên để BM25 tokenize chính xác.
- **normalize_unicode:** Chuẩn hóa tất cả ký tự Unicode về dạng NFC (Normalization Form Canonical Composition), đảm bảo các ký tự tiếng Việt có dấu được biểu diễn nhất quán.

6.2 Phân mảnh văn bản (Custom Chunker)

Sau khi chuẩn hóa, văn bản được chuyển qua custom chunker để cắt thành các chunk. Hệ thống có ba chunker riêng cho ba loại văn bản, vì mỗi loại có cấu trúc phân cấp khác nhau:

- **Luật:** Chương -> Mục -> Điều
- **Nghị định:** Chương -> Điều -> Khoản -> Điểm
- **Thông tư:** Điều -> Khoản

Mỗi chunk tương ứng với một đơn vị pháp lý hoàn chỉnh (thường là một Điều hoặc một Khoản). Chunker lưu metadata hierarchy với các trường chapter, diem, khoan, diem. Mỗi chunk được gán một định danh duy nhất theo mẫu: loại_văn_bản_số_năm__điều_x__khoản_y__c

Chương 7

LLM Integration

Hệ thống sử dụng hai mô hình ngôn ngữ lớn với vai trò khác nhau. Gemini 2.5 Flash là mô hình chính, đảm nhận các nhiệm vụ rewrite, grade, generate trong luồng xử lý. Mô hình này có free tier 250 request mỗi ngày, ngưỡng 1 triệu token, và hỗ trợ tiếng Việt tốt. OpenAI GPT-4o-mini là mô hình phụ, chỉ dùng cho đánh giá chất lượng, nhờ khả năng xuất JSON có cấu trúc ổn định.

Mỗi câu hỏi từ người dùng tiêu tốn trung bình bốn lần gọi Gemini. Nếu kể cả các bước fallback và sinh lại, con số này có thể lên đến bảy hoặc tám lần. Với free tier 250 request mỗi ngày, hệ thống phục vụ tối đa khoảng 62 câu hỏi mỗi ngày. Nếu vượt quá giới hạn, hệ thống tự động chuyển sang Gemini 2.0 Flash hoặc OpenAI GPT-4o-mini thông qua cơ chế fallback.

Việc gọi API Gemini được thực hiện thông qua LangChain. Mỗi lần gọi đều có timeout và retry với exponential backoff. Nếu Gemini trả về lỗi rate limit, hệ thống đợi một khoảng thời gian tăng dần (1 giây, 2 giây, 4 giây) rồi thử lại. Sau ba lần thử thất bại, hệ thống chuyển sang OpenAI.

Chương 8

Citation Pipeline

Citation pipeline đảm bảo mỗi câu trả lời có trích dẫn chính xác.

Khi LLM sinh câu trả lời ở nút generate, nó được yêu cầu xuất ra định dạng JSON với hai trường: answer là nội dung câu trả lời, citations là danh sách các trích dẫn. CitationExtractor nhận đầu ra này và thử phân tích cú pháp JSON trước. Nếu LLM trả về một đối tượng dict, nó được phân tích trực tiếp. Nếu LLM trả về văn bản tự do, bộ trích xuất dùng biểu thức chính quy để tìm các mẫu trích dẫn pháp luật.

Các mẫu biểu thức chính quy bao gồm: "Nghị định số X/YYYY/NĐ-CP", "Luật số X/YYYY/QH", "Thông tư số X/YYYY/TT", "Điều X", "Khoản Y", "Điểm Z". Bộ trích xuất cũng xử lý các định dạng lồng nhau như "[Nghị định 168/2024/NĐ-CP, Điều 6, Khoản 2, Điểm a]".

Sau khi trích xuất, CitationValidator kiểm tra từng trích dẫn có thực sự tồn tại trong context đã cung cấp hay không. Nó đối chiếu doc_id, điều, khoản, điểm với metadata của các chunk đã được retrieve. Nếu trích dẫn khớp với một chunk trong context, nó được đánh dấu hợp lệ. Nếu không khớp, nó được liệt kê vào validation_warnings.

Chương 9

Web Search Fallback

Khi corpus nội bộ không có đủ thông tin, hệ thống kích hoạt luồng tìm kiếm web dự phòng. Điều kiện kích hoạt là `relevance_score` dưới 0.4, hoặc số lần lặp đạt 3 mà vẫn chưa đạt điểm grade đủ cao, hoặc số lần rewrite đạt 2.

Nút `web_search` gọi thư viện `duckduckgo-search` để gửi truy vấn đến DuckDuckGo. Kết quả trả về là danh sách các URL cùng với tiêu đề và đoạn trích. Hệ thống lấy ba đến năm kết quả hàng đầu.

Nếu có kết quả, nút `generate_with_web_context` được gọi. LLM nhận các kết quả web làm ngữ cảnh và sinh câu trả lời. Một disclaimer được thêm vào cuối: "Nguồn: Web (chưa qua kiểm duyệt). Vui lòng xác minh trước khi áp dụng."

Nếu không có kết quả, nút `refuse_answer` được gọi. Hệ thống trả lời: "Tôi không tìm thấy thông tin trong cơ sở dữ liệu hiện có."

DuckDuckGo có thể bị chặn trên các địa chỉ IP trung tâm dữ liệu (Oracle Cloud, Vultr). Để phòng trường hợp này, hệ thống hỗ trợ SerpAPI làm fallback thông qua biến môi trường `SEARCH_BACKEND`.

Chương 10

Đánh Giá Hệ Thống RAGAS-lite [10]

Hệ thống được đánh giá trên bộ câu hỏi chuẩn (gold set) gồm 30 đến 50 câu. Mỗi câu trong gold set bao gồm câu hỏi, câu trả lời mong đợi, các trích dẫn chính xác, và một biến thể retrieval được ấn định. Gold set cũng bao gồm các câu nên bị từ chối (adversarial) để kiểm tra khả năng từ chối trả lời đúng lúc.

Năm chỉ số đánh giá được tính trên ba biến thể retrieval, tạo thành ma trận 3 nhân 5.

Citation Correctness là chỉ số quan trọng nhất. Nó đo tỷ lệ các câu trả lời có trích dẫn khớp chính xác với gold set. Đây là chỉ số xác định, không cần LLM judge. Hàm đánh giá so sánh từng trích dẫn trong câu trả lời với trích dẫn trong gold set: doc_id phải khớp, diu phải khớp, nếu gold set có khoan thì khoan phải khớp, nếu gold set có diem thì diem phải khớp. Nếu có ít nhất một trích dẫn khớp hoàn toàn, điểm là 1. Nếu không, điểm là 0.

Faithfulness đo mức độ câu trả lời trung thành với ngữ cảnh đã cung cấp. LLM judge nhận context và answer, trả về điểm 0-1.

Answer Relevancy đo mức độ câu trả lời liên quan đến câu hỏi. LLM judge nhận question và answer, trả về điểm 0-1.

Context Precision đo mức độ các ngữ cảnh được retrieve có chứa câu trả lời đúng và được xếp hạng đúng thứ tự. LLM judge nhận question và contexts, trả về điểm 0-1.

Context Recall đo mức độ các ngữ cảnh được retrieve có chứa đủ thông tin để trả lời câu hỏi. LLM judge nhận question, expected_answer, và contexts, trả về điểm 0-1.

LLM judge được cố định là OpenAI GPT-4o-mini cho tất cả các biến thể, bất kể biến thể đó dùng LLM nào để sinh câu trả lời. Điều này ngăn chặn circularity: nếu cùng LLM vừa sinh câu trả lời vừa đánh giá, điểm số sẽ thiên vị.

Điểm số được phân tích bằng regex pattern `r"[-+]?*??"` để trích xuất số từ câu trả lời của LLM judge, thay vì dùng `float()` trực tiếp.

Chương 11

Bảo Mật và PDPA

Hệ thống tuân thủ Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân. Các yêu cầu cụ thể bao gồm: không lưu trữ thông tin nhận dạng cá nhân ngoài thread_id, chính sách lưu trữ tối đa 30 ngày cho các cuộc hội thoại với cơ chế xóa tự động, xác thực admin bằng Bearer token thông qua biến môi trường ADMIN_TOKEN, tất cả API endpoint admin và ingest yêu cầu Bearer token, và không ghi log các truy vấn có chứa thông tin cá nhân.

Tốc độ API bị giới hạn ở 6 request mỗi phút mỗi địa chỉ IP, tương thích với giới hạn 10 RPM của Gemini Free Tier và sử dụng slowapi với header X-Real-IP để phát hiện địa chỉ IP thực.

Chương 12

Domain Model

Hệ thống có các thực thể chính sau đây. Document đại diện cho một văn bản pháp luật đã được ingest, bao gồm doc_id, title, doc_type (LUAT, NGHI_DINH, THONG_TU), issue_date, status (HIEU_LUC, HET_HIEU_LUC, CHUA_HIEU_LUC), source_url, pdf_path, file_hash (để khử trùng lặp), và chunk_count. Chunk đại diện cho một đoạn văn bản đã được cắt ra từ Document, bao gồm chunk_id, doc_id, content, hierarchy (dieu, khoan, diem, chapter), page, embedding, và status. Conversation đại diện cho một phiên hội thoại, bao gồm thread_id, user_id, title, và danh sách Message. Message chứa nội dung tin nhắn, vai trò (user, assistant, system), và tham chiếu đến chunks và citations. Query lưu trữ thông tin về một câu hỏi cụ thể, bao gồm raw_query, rewritten_query, intent, top_k_chunks, answer, citations, web_used. Citation là một trích dẫn có cấu trúc: doc_title, dieu, khoan, diem, page, snippet, status.

Chương 13

Concurrency và Thread Safety

ChromaDB sử dụng SQLite nội bộ với cơ chế persistent client. SQLite mặc định có thời gian chờ busy timeout ngắn và không bật WAL, dẫn đến xung đột khi nhiều tiến trình truy cập đồng thời. FastAPI với công thức mặc định workers = 2 nhân CPU cộng 1 sẽ tạo ra 13 worker trên máy i5, dẫn đến 13 tiến trình cùng đọc ghi ChromaDB. Kết quả là treo 16 phút, segmentation fault, hoặc hỏng vĩnh viễn cơ sở dữ liệu.

Giải pháp là chạy Uvicorn với `--workers 1` và bật WAL mode cho SQLite. WAL (Write-Ahead Log) cho phép đọc và ghi đồng thời mà không khóa toàn bộ cơ sở dữ liệu. Trade-off là throughput chỉ đạt khoảng 2.5 query mỗi phút thay vì 13 query mỗi phút nếu dùng multi-worker, nhưng tính toàn vẹn dữ liệu được đảm bảo.

SQLite cho state database sử dụng SQLAlchemy với `connect_args check_same_thread=False`, `timeout=30`, và bật WAL mode qua event listener. Điều này cho phép FastAPI xử lý đồng thời các request ghi state khác nhau.

Chương 14

Ingest Pipeline

Khi có văn bản luật mới, nhà quản trị chạy script `ingest_corpus.py`. Pipeline thực hiện các bước sau: parse PDF bằng PyMuPDF, chunk theo loại văn bản, embed các chunk mới bằng `embedding_service.embed_passages`, thêm vào ChromaDB với chế độ append (không xóa chunk cũ), tái xây dựng toàn bộ chỉ mục BM25, và cập nhật bảng Document với `chunk_count` và `updated_at`.

Việc cập nhật BM25 phải xây dựng lại từ đầu (full rebuild) vì thư viện `rank_bm25` không hỗ trợ thêm dữ liệu gia tăng. Thao tác này mất vài giây cho 30 văn bản nhưng sẽ tăng tuyến tính với kích thước corpus.

Khi có văn bản hết hiệu lực, script `check_expired.py` (theo kế hoạch trong `08-bao-tri.md`) quét toàn bộ corpus bằng cách đọc metadata `expiry_date` của từng chunk trong ChromaDB, so sánh với ngày hiện tại. Văn bản có `expiry_date` trong quá khứ được liệt kê là đã hết hiệu lực. Văn bản có `expiry_date` trong vòng 30 ngày tới được liệt kê là sắp hết hiệu lực.

Chương 15

Ràng Buộc Phần Cứng

Hệ thống chạy trên máy Intel Core i5-1035G1 với 4 nhân 8 luồng, 19GB RAM, card đồ họa NVIDIA MX330 2GB VRAM. MX330 không đủ mạnh để chạy embedding inference ổn định (Out of Memory khi dùng CUDA), do đó mọi tác vụ embedding đều chạy trên CPU. Mô hình reranker đã bị loại bỏ khỏi thiết kế vì lý do này.

Embedding time khoảng 300 đến 500 mili giây cho mỗi query sau lần khởi tạo đầu tiên nhờ bộ nhớ đệm. Retrieval time khoảng 2 giây cho mỗi query. Latency P50 của toàn bộ hệ thống dưới 12 giây. Latency P95 dưới 20 giây, dựa trên bốn lần gọi LLM mỗi lần khoảng 3 giây cộng retrieval 2 giây.

Chương 16

Kiến Trúc Giao Diện

Giao diện chat có bốn thành phần chính. ChatWindow chứa toàn bộ khung chat với streaming. MessageBubble hiển thị tin nhắn người dùng và trợ lý với định dạng Markdown. CitationBadge hiển thị badge cho mỗi trích dẫn dạng "[NĐ 168, Điều 6]" và có thể mở rộng để xem chi tiết. StreamingResponse xử lý sự kiện SSE để cập nhật câu trả lời theo thời gian thực.

Câu trả lời hiển thị dưới dạng Markdown với các phần: nội dung câu trả lời, danh sách nguồn tham khảo ở cuối với citation có thể click để mở rộng xem snippet, disclaimer ở cuối cùng, và trạng thái hiệu lực của từng văn bản được trích dẫn.

Chương 17

Giao Diện API

Hệ thống cung cấp các REST API endpoint sau:

Bảng 17.1. Danh sách REST API Endpoint

Method	Path	Mô tả	Auth
POST	/api/chat	Gửi câu hỏi, trả về streaming	Không
POST	/api/chat/stream	SSE streaming response	Không
GET	/api/conversations/{thread_id}	Lấy lịch sử hội thoại	Không
GET	/api/search?q=...	Tìm kiếm từ khóa trực tiếp	Không
POST	/api/ingest	Upload PDF (multipart)	Admin
GET	/api/admin/pending	Danh sách HITL pending	Admin
POST	/api/admin/approve/{review_id}	Phê duyệt HITL	Admin
POST	/api/admin/reject/{review_id}	Từ chối HITL	Admin
POST	/api/eval/run	Chạy evaluation	Admin
GET	/api/eval/results	Xem kết quả đánh giá	Không
GET	/api/health	Health check	Không
GET	/api/docs	Swagger UI (tự động)	Không

Request chat có dạng JSON với trường query (câu hỏi) và thread_id tùy chọn. Response trả về thread_id, query_id, answer dưới dạng Markdown, danh sách citations có cấu trúc, disclaimer, citation_statuses, validation_warnings, web_used, và latency_ms.

Chương 18

Kiến Trúc Kiểm Thử

Hệ thống có ba lớp kiểm thử. Unit test kiểm tra từng module độc lập: parser PDF, chunker, embedding service, citation extractor, BM25 index. Mục tiêu coverage ≥ 70

Evaluation pipeline chạy qua script `run_eval.py`: tải gold set, chạy từng câu hỏi qua toàn bộ pipeline RAG (bao gồm generation, không chỉ retrieval-only), thu thập citations và answers, gọi GPT-4o-mini làm judge cho 4 metrics RAGAS-lite, tính Citation Correctness bằng exact match, và xuất báo cáo JSON. Ba biến thể được đánh giá độc lập: V1 (chỉ Dense), V2 (chỉ BM25), V3 (Hybrid), tạo thành ma trận $3 \text{ biến thể} \times 5 \text{ metrics}$ để so sánh.

Chương 19

Tổng Kết

Hệ thống kết hợp năm kỹ thuật chính để giải quyết bài toán tra cứu pháp luật giao thông: hybrid retrieval kết hợp dense và BM25 để tối ưu truy xuất tiếng Việt, LangGraph state machine để điều phối luồng xử lý tự động với loop guard chống vòng lặp vô hạn, citation pipeline với validation để đảm bảo trích dẫn chính xác, RAGAS-lite với năm chỉ số để đánh giá chất lượng, và cơ chế web fallback với disclaimer để xử lý các câu hỏi ngoài phạm vi.

Tài liệu tham khảo

- [1] Chính phủ. *Nghị định 168/2024/NĐ-CP quy định xử phạt vi phạm hành chính về trật tự, an toàn giao thông trong lĩnh vực giao thông đường bộ*. 2024.
- [2] Quốc hội. *Luật Trật tự, An toàn giao thông đường bộ số 36/2024/QH15*. 2024.
- [3] Chính phủ. *Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân*. 2023.
- [4] Vaswani, A., et al. *Attention Is All You Need [4]*. NeurIPS, 2017.
- [5] Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
- [6] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR, 2009.
- [7] Robertson, S., & Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 2009.
- [8] Wang, L., et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. arXiv:2212.03533, 2022.
- [9] LangChain. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. 2024.
- [10] Shahul, E. S. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. 2024.
- [11] Google. *Gemini API Documentation: Pricing and Rate Limits*. 2026.
- [12] Chroma. *ChromaDB Documentation: Persistent Client Concurrency Issues*. 2024.
- [13] CTU-LinguTechies. *VN-Law-Advisor*. GitHub Repository. 2024.
- [14] Viblo. *Xây dựng Hệ thống Agentic RAG Pháp luật Giao thông*. 2024.